

TESTS

- **Test funzionali**: la mia applicazione fa quello che deve fare?
- **Test delle prestazioni**: lo fa in modo rapido ed efficiente?
- **Test di accessibilità**: funziona bene con i servizi di accessibilità?
- **Test di compatibilità**: funziona bene su ogni dispositivo e livello di API?

- Ad esempio:
 - Operazioni matematiche che utilizzano numeri negativi, zero e condizioni al contorno
 - Tutti i possibili errori di connessione alla rete
 - Dati corrotti, come JSON malformato
 - Simulare il disco pieno durante il salvataggio in un file
 - Oggetti ricreati nel mezzo di un processo, come quando un dispositivo viene ruotato

DA FARE nei seguenti casi

- **ViewModel**
- **Data layer**, in particolare i **repository**
 - che devono essere indipendenti dalla piattaforma Android
 - i **test double** sostituiscono i data base e le fonti dati remote con oggetti fake o mock
- **Altri layer** indipendenti dalla piattaforma Android
 - esempio: il livello di logica (Domain Layer)
- **Classi di utilità**
 - esempio: la manipolazione di stringhe ed algoritmi (ad esempio, conversione sistemi di misura)

• **TEST UNITÀ**: comportamento metodo/classe isolato dal sistema

• **TEST INTEGRAZIONE**: collaborazione tra componenti

- integrazione con **Service** o **Content Provider**: occorre verificare che tali componenti si comportino in modo corretto con la app
- interazioni tra una **View** e un **ViewModel**
- interazione corretta tra un **ViewModel** e il **Repository**
- interazione fra tutti i **layer** del modulo

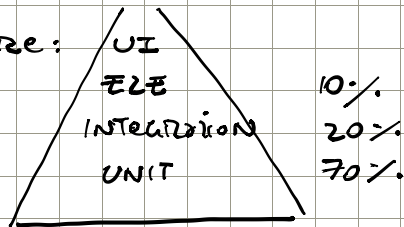
• **TEST END-TO-END**: funzionamento sistema in possibile scenario di utilizzo
UI → database → API

DA FARE:

- **Screen UI test**
 - Verificano le interazioni critiche dell'utente in una singola schermata
 - Eseguono azioni come il clic sui **pulsanti**, il riempimento di **form** e la verifica degli stati visibili all'utente (selezione/deselezione di item)
 - Una classe di test per schermata è un buon punto di partenza
- **User flow test o Navigation test**
 - Verificano la correttezza dei percorsi più comuni
 - Sono test semplici, utili per verificare la presenza di arresti anomali durante l'esecuzione
 - Testate solo flussi significativi: evitare UI test ridondanti o legati a dettagli estetici

• **TEST UI**: comportamento UI

%. **TEST DA ESEGUIRE**:



Espresso

UI Automator 2

App Crawler

Dove testare:

- Fisico: massima fedeltà, lento
- virtuale: equilibrio fedeltà - velocità (AS)
- JVM

ARCHITETTURA TESTABILE:

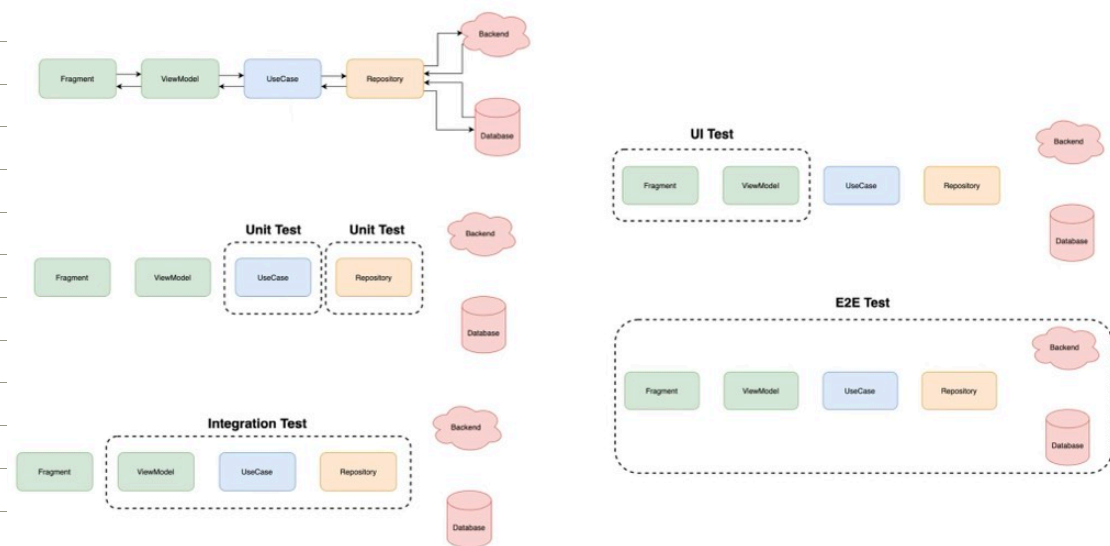
Permette di testare varie parti app in modo isolato

4. DISACCOPPIAMENTO

- Divisione in Presentazione, Dominio, Dati
- No **Cosica** in Activity / Fragment
 ↳ nei ViewModel

- Rendere dipendenze sostituibili: → **INTERFACCIA** invece di implementazioni concrete

DEPENDENCY INVERSION



Per isolare test → **TEST DOUBLES**: agiscono come se fossero componenti reali

Oggetto da testare potrebbe dependere da altri per funzionare
(es ViewModel dipende da Repository di dati)

• Fake

- Sono oggetti che hanno comportamenti **analoghi** a quelli reali, ma usano 'scorciatoie' per migliorare le prestazioni
- Esempio: database in memoria vs accesso API esterna
- Sono implementati a mano

• Mock

- Sono oggetti che hanno un comportamento **predefinito** e non hanno quindi implementazioni funzionanti
- Servono per testare il comportamento del codice piuttosto che il suo output
- Esempio: un oggetto mock potrebbe essere programmato per restituire un valore specifico quando viene chiamato con determinati argomenti
- Sono creati con framework appositi (ad esempio Mockito)

Posizione:

- ✓ com.example.pushapp (androidTest)
- ✓ ExampleInstrumentedTest
- ✓ com.example.pushapp (test)
- ✓ ExampleUnitTest

eseguiti su dispositivi reali/virtuali (necessitano Android)

→ integration/e2e/ui/... dipendente

eseguiti su macchina locale → **JUnit**
→ UNIT TEST

DA TESTARE:

cartella test/

MA

1. UNIT TEST (test locali)

JUnit

a mano

- ViewModel: stato UI cambia correttamente in base agli input (comunicamento dati, successo, errore)
- Utility/Domain classes: classi che fanno calcoli, conversioni, validazioni (es. validatore mail)
- Repository: verificare come gestiscono dati (modellando sorgenti dati → DB e API)

+ test mancanza connessione / casi limite

Se ViewModel dipende da Repository → Mock / Fake per simulare il comportamento

2. UI/E2E (test strumentati):

cartella AndroidTest/

Espresso

- Percorsi critici: Login → Home → ... (flussi principali)
- Interazione UI: cliccando bottone succede l'azione prevista

... e classi che dipendono da Android

Per testare scenari complessi DB/API → Mock

Eseguire con AndroidJUnitRunner (automatico)

ToDo

1. **Disaccoppia il codice:** Assicurati di aver spostato la logica fuori dalle Activity e dentro i ViewModel. Le Activity devono essere solo "punti di ingresso". Se il codice è tutto nell'Activity, è difficilissimo da testare.
2. **Copri i ViewModel (Unit Test):** Scrivi test per ogni funzione pubblica dei tuoi ViewModel.
 - Obiettivo: Coprire casi normali e "casi limite" (es. input nulli, errori di rete simulati).
3. **Crea 1-2 UI Test completi (End-to-End):** Simula un utente che apre l'app e compie l'azione principale (es. "Login → Vedo la Dashboard"). Questo dimostra che l'integrazione tra le parti funziona.
4. **Robolectric (Opzionale ma utile):** Se hai bisogno di testare logica che dipende leggermente da Android (es. contesti, risorse) ma vuoi farlo velocemente in locale, puoi usare Robolectric, ma ricorda che non sostituisce i test su device reale.

es. AppNavigationTest.java

1/2 scenari più importanti:

es.

- Username → psw → login → Home
- Home → profilo
- Home → workout

Invece di creare file e cartelle a mano, fai così:

1. Apri la classe che vuoi testare (es. apri `ProfileActivity.java` o una tua classe di logica).
2. Clicca sul nome della classe.
3. Premi la combinazione: `CTRL + SHIFT + T` (Windows) o `CMD + SHIFT + T` (Mac).
4. Seleziona **Create New Test...**
5. Scegli **JUnit4** e spunta i metodi che vuoi testare.
6. Android Studio creerà automaticamente il file nella cartella giusta (`test/java/...`) con la struttura già pronta.

Tu dovrai solo riempire il contenuto del metodo `@Test` con la logica:

```
Java
// Il file lo crea Android Studio, tu scrivi solo questo dentro:
@Test
public void calcolaMedia_isCorrect() {
    // 1. Prepara i dati
    // 2. Chiama il metodo
    // 3. Verifica il risultato (Assert)
}
```

<https://developer.android.com/studio/test/other-testing-tools/espresso-test-recorder?hl=it>

① EspressoTestRecorder per generare codice automaticamente

```
import static androidx.test.espresso.matcher.ViewMatchers.withText;

@RunWith(AndroidJUnit4.class)
public class LoginFlowTest {

    // Questa regola lancia l'Activity specificata prima di ogni test
    // Qui diciamo di partire dalla LoginActivity
    @Rule
    public ActivityScenarioRule<LoginActivity> activityRule =
        new ActivityScenarioRule<>(LoginActivity.class);

    @Test
    public void testLoginSuccess() {
        // 1. TROVA la vista e AGISCI: Scrivi username
        onView(withId(R.id.et_username))
            .perform(typeText("utente_test"), closeSoftKeyboard()); // [cite: 808]

        // 2. Scrivi password
        onView(withId(R.id.et_password))
            .perform(typeText("password123"), closeSoftKeyboard());

        // 3. Clicca il bottone Login
        onView(withId(R.id.btn_login))
            .perform(click()); // [cite: 809]

        // 4. VERIFICA il risultato:
        // Controlliamo che appaia una vista tipica della MainActivity.
        // Ad esempio, controlliamo se c'è la scritta "Hello World!" che hai nel
        onView(withText("Hello World!"))
            .check(matches(isDisplayed())); // [cite: 810, 813]
    }
}
```

3.

TEST AUTOMATICI

- **App Crawler:** È uno strumento che "testa automaticamente le azioni dell'utente senza dover scrivere o mantenere alcun codice". Naviga autonomamente nell'app cercando crash o problemi di interfaccia. @
- **Monkey:** È uno "strumento a riga di comando che invia un flusso pseudo-casuale di eventi utente all'emulatore o dispositivo". È utile per fare "stress test" (cioè vedere se l'app regge a un uso frenetico e casuale senza crashare), simulando click, tocchi e gesture a caso. @
- **Espresso Test Recorder:** È uno strumento integrato che "permette di registrare test UI". Tu usi l'app manualmente (clicchi i bottoni), lui registra le tue azioni e genera automaticamente il codice del test Espresso per te. @

} da usare come stress-test finale

4.

TEST SU DISPOSITIVI REALI: Testare compatibilità su device diversi

Questi servizi servono per lanciare i tuoi test (automatici o manuali) su una vasta gamma di dispositivi reali, che magari non possiedi fisicamente.

- **Firebase Test Lab:** Permette di "testare simultaneamente la app su molti dispositivi Android e molte differenti configurazioni" (come orientamento, dimensioni schermo, versione API) sfruttando i centri dati di Google. I risultati includono log e dettagli sui malfunzionamenti. @ @

Testare: No connessione, batteria scarica, chiamate in arrivo

<https://github.com/android/testing-samples>